

Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

КУРСОВА РОБОТА

з дисципліни «Об'єктно-орієнтоване програмування»

РОЗРОБКА БАЗИ ДАНИХ ДЛЯ ПІДТРИМКИ РОБОТИ ФІРМИ З ПРОДАЖУ
АВТОМОБІЛІВ, ЩО БУЛИ У ВИКОРИСТАННІ

Студентки __2__ курсу AI-241 групи
Спеціальності F3 – «Комп'ютерні науки»
Горбатової В.Ю.
(прізвище та ініціали)

Керівник доцент Годовиченко М.А.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала _____

Кількість балів: _____

Оцінка: ECTS _____

Члени комісії

_____	_____
(підпис)	(прізвище та ініціали)
_____	_____
(підпис)	(прізвище та ініціали)
_____	_____
(підпис)	(прізвище та ініціали)

м. Одеса – 2026 рік

АНОТАЦІЯ

Горбатова В.Ю. Розробка інформаційної системи спортивного клубу: курсова робота з дисципліни «Об'єктно-орієнтоване програмування» за спеціальністю «122 Комп'ютерні науки» / Горбатова В.Ю.; керівник Годовиченко М.А. – Одеса : Нац. ун-т «Одес. політехніка», 2026. – 46 с.

Курсова робота містить основну текстову частину на 46 сторінках, список використаних джерел з 1 найменування на 1 сторінці.

У курсовій роботі розроблено інформаційну систему спортивного клубу для автоматизації обліку клієнтів, тренерів, тренувань та записів на заняття. Проведено аналіз предметної області, визначено користувачів системи та їхні потреби, спроектовано концептуальну та логічну модель бази даних. Реалізовано серверну частину застосунку на базі Java Spring Boot із використанням PostgreSQL, Spring Data JPA та REST API. Забезпечено механізми валідації даних, обробки виняткових ситуацій, пошуку, аналітики та моніторингу стану системи. Розроблена система дозволяє автоматизувати процес формування розкладу тренувань, управління клієнтською базою та реєстраціями на заняття, підвищуючи ефективність роботи спортивного клубу.

Ключові слова: спортивний клуб, PostgreSQL, Spring Boot, REST API, Hibernate, DTO, Swagger, Actuator.

ЗМІСТ

ЗАВДАННЯ НА КУРСОВУ РОБОТУ	5
ВСТУП.....	6
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ	8
1.1 Опис предметної області, для якої створюється база даних	8
1.2 Опис користувачів системи та їх історії (User Story).....	9
1.3 Детальний опис функціоналу бази даних, що пропонується	11
Висновки до розділу 1	16
2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	17
2.1 Проєктування структури даних	17
2.2 Опис архітектури застосунку	18
2.3 Опис REST API.....	19
Висновки до розділу 2	23
3 РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ	24
3.1 Конфігурація системи (Configuration Layer)	24
3.2 Моделі даних (Entity-класи та DTO)	25
3.3 Репозиторії (Repository Layer)	27
3.4 Сервіси (Service Layer)	28
3.5 Контролери (Controller Layer).....	30
3.6 Глобальна обробка виняткових ситуацій (Exception Handling)	33
Висновки до розділу 3	36
4 Тестування та налагодження.....	37
4.1 Інструменти тестування та налагодження.....	37
4.2 Структура тестової колекції в Postman.....	37
4.3 Результати тестування основних функцій системи.....	40
Висновки до розділу 4	43
ЗАГАЛЬНІ ВИСНОВКИ	44

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	46
---------------------------------	----

Національний університет «Одеська політехніка»
Навчально-науковий інститут комп'ютерних систем
Кафедра інформаційних систем

ЗАВДАННЯ НА КУРСОВУ РОБОТУ

студенту Горбатової Вікторії Юріївні

група AI-241

1. Тема роботи: розробка бази даних для підтримки роботи фірми з продажу автомобілів, що були у використанні.

2. Термін здачі студентом закінченої роботи

08.06.2026

3. Початкові дані до проекту (роботи) Варіант 7: Завдання: Розробити базу даних та веб-орієнтований сервіс для автоматизації роботи спортивного клубу. Вхідні дані: відомості про клієнтів (ПІБ, email, телефон), тренерів (ПІБ, спеціалізація, email), тренування (назва, опис, дата проведення, тривалість), типи тренувань (назва, опис), записи на тренування (клієнт, тренування, дата реєстрації, статус). Вихідні дані: інформація про клієнтів, тренерів і тренування, розклад занять, списки учасників тренувань, дані про записи клієнтів, статистичні та аналітичні звіти щодо діяльності клубу.

Функціонал: ведення обліку клієнтів і тренерів спортивного клубу; створення, редагування та видалення тренувань; управління записами клієнтів на заняття; класифікація тренувань за типами; пошук клієнтів, тренерів і тренувань; формування аналітичних звітів; контроль цілісності даних і дотримання бізнес-правил; моніторинг технічного стану системи.

4. Зміст розрахунково-пояснювальної записки (перелік питань, які належить розробити) Проаналізувати предметну область «Спортивний клуб», визначити користувачів системи та її функціональні можливості. Виконати проектування бази даних, визначити основні сутності та зв'язки між ними. Обрати та обґрунтувати програмні засоби реалізації. Створити структуру бази даних і механізми забезпечення цілісності даних. Реалізувати SQL-запити та REST API для роботи з клієнтами, тренерами, тренуваннями, типами тренувань і записами. Розробити механізми валідації, обробки винятків та аналітики. Налаштувати права доступу користувачів, провести тестування системи та проаналізувати результати її роботи.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень) _____

Завдання видано

3.06.2026

(підпис викладача)

Завдання прийнято до виконання 3.06.2026

(підпис студента)

ВСТУП

У сучасних умовах розвитку інформаційних технологій та автоматизації бізнес-процесів інформаційні системи та веб-орієнтовані рішення є невід’ємною складовою будь-якої організації, що прагне ефективно управляти інформацією. В умовах ринку надання спортивних та оздоровчих послуг фітнес-клуби стикаються з великими обсягами даних: інформацією про клієнтів, тренерський склад, розклад занять, категорії тренувань та реєстраційні записи на відвідування. Без належного системного підходу до ведення обліку виникають проблеми дублювання даних, накладки у графіках занять, складність у швидкому пошуку необхідних записів та відсутність контролю за виконанням щоденних бізнес-процесів.

Створення спеціалізованої веб-системи (REST API) дозволяє централізовано зберігати інформацію, автоматизувати процеси обліку, спростити доступ до даних та підвищити точність їх обробки. Розроблена система надає можливість швидко отримувати необхідну статистику для прийняття управлінських рішень, контролювати стан поточних записів на заняття, ефективно управляти розподілом тренерського навантаження, забезпечувати гнучку валідацію вхідних даних та гарантувати безпеку шляхом централізованої обробки виняткових ситуацій.

Актуальність розробки програмного забезпечення для сфери фітнес-послуг обумовлена необхідністю підвищення ефективності роботи адміністрації та тренерів, зменшення витрат часу на планування розкладу та підвищення рівня обслуговування клієнтів. Сучасні спортивні центри потребують інтегрованих, масштабованих рішень на базі сучасного стеку технологій (Java, Spring Boot, PostgreSQL), які дозволяють контролювати всі аспекти діяльності: від персонального обліку відвідувачів до детального аналізу популярності фітнес-напрямків і автоматичного моніторингу працездатності додатку.

Мета роботи полягає у проектуванні, програмній реалізації та документуванні функціональної веб-системи автоматизації та аналітики для фітнес-клубу. Система повинна забезпечувати надійне керування даними клієнтів, тренерів і тренувань, надавати інтегровані інструменти для повнотекстового

пошуку і збирання аналітичних метрик у режимі реального часу, а також гарантувати наявність повної, інтерактивної та деталізованої документації інтерфейсу програмування за допомогою інструментів OpenAPI/Swagger.

Реалізація поставленої мети дозволить оптимізувати внутрішні процеси менеджменту спортивного центру, забезпечити точність, цілісність і зв'язаність даних на рівні архітектури бази даних, спростити взаємодію персоналу з інформацією та підвищити загальну ефективність управління якістю послуг.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

У цьому розділі проаналізовано діяльність сучасного спортивного клубу та визначено основні процеси, що потребують автоматизації. На основі опису користувачів і їхніх потреб сформульовано вимоги до функціональності системи. Сформовано постановку задачі, яка визначає склад сутностей, зв'язків і функцій майбутнього backend-сервісу та його бази даних.

1.1 Опис предметної області, для якої створюється база даних

Предметною областю даного проєкту є діяльність спортивного клубу (фітнес-центру), який надає послуги з організації та проведення різнопланових тренувань. Основна мета роботи такої організації полягає в забезпеченні клієнтів якісними спортивними програмами, координації роботи тренерського складу, формуванні актуального розкладу та обліку відвідуваності. Посередницьку й організаційну функцію виконує інформаційна система, яка пов'язує докупки клієнтів клубу, тренерів, розклад занять та специфіку напрямків тренувань.

У предметній області фітнес-послуг виникає потреба у збереженні великої кількості різнорідних даних – відомостей про клієнтів, тренерів, детальний опис та тривалість тренувань, типи занять, а також реєстраційні записи на тренування. Усе це ускладнює роботу адміністрації, особливо за умов, коли інформація зберігається у вигляді паперових журналів, розрізнених файлів або таблиць Excel.

До основних проблем, що існують у даній предметній області, належать: неефективне та фрагментарне зберігання даних, складність швидкого пошуку потрібного заняття у розкладі, дублювання відомостей (наприклад, накладки у графіку тренера), ризик запису клієнта на заліковий час, що вже зайнятий, а також повна відсутність єдиного інструменту для збирання аналітики щодо завантаженості залів і популярності тренерів. Крім того, адміністратори витрачають багато часу на ручний контроль статусів записів клієнтів, що знижує

продуктивність роботи фірми та підвищує ймовірність виникнення помилок через людський фактор.

Для вирішення зазначених проблем необхідно створити веб-орієнтований backend-сервіс та реляційну базу даних, які забезпечать централізоване, структуроване, надійне зберігання інформації, а також нададуть можливість миттєвого пошуку, фільтрації, редагування та аналізу даних. Така система стане основою цифрової інфраструктури спортивного клубу, автоматизує процеси реєстрації нових відвідувачів, ведення розкладу та обліку записів на заняття.

База даних повинна містити відомості про тренерів (їхні ПІБ, спеціалізацію, контактний email), клієнтів клубу (ПІБ, телефон, email), тренування (назву, опис, дату та час проведення, тривалість у хвилинах). Окрему роль відіграють сутності типів тренувань (назва напрямку, опис) та реєстраційні записи, що фіксують факт запису конкретного клієнта на певне заняття із зазначенням статусу (наприклад, активний чи скасований).

Запровадження інформаційної системи на основі розробленого REST API значно полегшить роботу користувачів. Система дозволить автоматично валідувати вхідні дані (перевіряти довжину імен, унікальність email, коректність телефонів та дат), швидко знаходити тренування чи клієнтів за текстовим запитом, відстежувати стан кожної реєстрації та збирати аналітичні звіти для керівництва в один клік. Керівництво зможе миттєво оцінювати ефективність роботи тренерів на основі кількості проведених ними занять та аналізувати загальну чисельність активної клієнтської бази.

Таким чином, розробка backend-сервісу для підтримки діяльності спортивного клубу є необхідною умовою для оптимізації роботи підприємства. Вона дозволить мінімізувати обсяг ручної роботи, підвищити швидкість обробки заявок, забезпечити цілісність, зв'язаність та достовірність даних, а також покращити загальну якість обслуговування відвідувачів.

1.2 Опис користувачів системи та їх історії (User Story)

Інформаційна система, що розробляється для управління спортивним клубом, має декілька основних категорій користувачів (ролей). Кожен тип користувача має власні цілі, обов'язки та потреби у функціоналі серверної частини. Ниче наведено опис основних ролей користувачів та їхніх історій використання (User Story), які визначають вимоги до функціональності бази даних, DTO-класів та REST-ендпоінтів.

Роль: Адміністратор клубу (Менеджер).

Адміністратор є ключовим користувачем системи, який координує щоденну діяльність закладу, веде облік персоналу та формує сітку розкладу.

1. Як адміністратор, я хочу мати змогу реєструвати нових тренерів та клієнтів у системі, щоб розширювати базу даних організації.

2. Як адміністратор, я хочу створювати нові тренування, вказуючи назву, тривалість та прив'язуючи їх до конкретного тренера, щоб гнучко формувати загальний розклад клубу.

3. Як адміністратор, я хочу мати можливість оновлювати контактні дані клієнтів або спеціалізацію тренерів у разі зміни їхньої інформації.

4. Як адміністратор, я хочу мати змогу видалити запис про тренування чи тип заняття, якщо воно було скасоване або більше не проводиться в клубі.

5. Як адміністратор, я хочу мати інструмент повнотекстового пошуку, щоб швидко знаходити профілі клієнтів або тренерів за ключовими словами без ручного перегляду всього списку.

Роль: Клієнт клубу (Відвідувач).

Клієнт взаємодіє із системою з метою планування своїх персональних занять та відстеження власної активності.

1. Як клієнт, я хочу мати змогу записатися на обране тренування, щоб зафіксувати за собою місце в групі чи індивідуальному графіку.

2. Як клієнт, я хочу переглядати актуальний розклад усіх тренувань, щоб обрати найбільш зручний для себе час.

3. Як клієнт, я хочу бачити інформацію про тренера, який проводить заняття, включаючи його спеціалізацію та контакти.

4. Як клієнт, я хочу мати можливість скасувати свій запис на тренування, якщо мої плани змінилися, щоб звільнити місце для інших відвідувачів.

5. Як клієнт, я хочу бачити список усіх тренувань, які я збираюся відвідати, для ефективного планування свого часу.

Роль: Керівник (Директор спортивного клубу).

Керівник використовує систему як інструмент верхньорівневого моніторингу та збирання аналітики для оцінки фінансових показників та успішності бізнесу.

1. Як керівник, я хочу бачити загальну статистику щодо кількості активних клієнтів та загальної кількості створених занять, щоб оцінювати темпи розвитку клубу.

2. Як керівник, я хочу отримувати звіт про кількість тренувань, згрупованих за їхніми типами (напрямами), щоб розуміти, які види занять приносять найбільший рух і є затребуваними.

3. Як керівник, я хочу бачити рейтинг найпопулярніших тренерів, відсортований за кількістю проведених ними занять, для розрахунку заробітної плати чи виплати бонусів.

4. Як керівник, я хочу мати доступ до технічних метрик сервісу та стану його працездатності (Health Check), щоб бути впевненим у стабільній роботі системи.

Таким чином, користувачі системи мають різні потреби, але всі вони взаємопов'язані спільною метою – забезпечити ефективний та безпомилковий процес організації спортивних занять. Розроблений backend-сервіс повинен повністю задовольняти ці потреби через надання чітких, задокументованих та безпечних REST-ендпоінтів.

1.3 Детальний опис функціоналу бази даних, що пропонується

На основі аналізу предметної області сформульовано мету створення сервісу — автоматизація процесів планування розкладу, обліку відвідувачів та ведення аналітики спортивного клубу. Для реалізації цієї мети розробляється інформаційна система, яка забезпечить централізоване зберігання даних, швидкий доступ до них

через архітектуру REST, механізми суворої валідації полів (Bean Validation), а також надійний контроль бізнес-процесів за допомогою шару сервісів.

Перелік завдань, які необхідно вирішити:

1) створення реляційної структури бази даних, що охоплює сутності: Trainer, Member, Workout, Registration, WorkoutType;

2) забезпечення цілісності та зв'язаності даних на рівні СУБД PostgreSQL за допомогою первинних, зовнішніх ключів та таблиць мапінгу зв'язків ManyToMany;

3) розробка шару DTO-класів для чіткого розмежування внутрішніх JPA-сутностей та вхідних/вихідних JSON-пакетів даних;

4) реалізація суворих правил валідації вхідних запитів (перевірка довжини рядків, обов'язковості полів, форматів email та телефонів);

5) побудова системи глобальної обробки помилок з поверненням єдиного стандартизованого JSON-об'єкта відповіді про помилку (ApiErrorResponse);

6) впровадження бізнес-правил, що блокують створення дублікатів, видалення сутностей з активними зв'язками або логічно некоректні операції;

7) інтеграція модулів аналітики, повнотекстового пошуку та збирання технічних метрик додатку (Actuator).

Перелік функцій, які має підтримувати інформаційна система:

1) робота з тренерами (Trainers):

- створення профілю тренера з перевіркою обов'язковості полів та унікальності email;

- отримання повного списку або інформації про конкретного тренера за його ID;

- оновлення даних тренера (дозволених для редагування полів);

- видалення профілю тренера (якщо з ним не пов'язані активні бізнес-об'єкти);

- отримання розкладу занять, закріплених за конкретним тренером;

2) робота з клієнтами (Members):

- реєстрація клієнта із суворою валідацією імені, унікального email та телефонного номера;

- перегляд списку зареєстрованих клієнтів та пошук конкретної особи за ідентифікатором;

- модифікація персональної або контактної інформації клієнта;

- повне видалення профілю клієнта з системи;

- отримання списку тренувань, на які записаний даний клієнт;

3) робота з тренуваннями (Workouts):

- створення нового тренування у розкладі із зазначенням назви, опису, дати проведення та тривалості;

- отримання актуальної сітки розкладу та перегляд детальних параметрів окремого заняття;

- коригування часу, тривалості чи опису тренування через PUT-запити;

- видалення тренування з розкладу;

- вибірка занять за ідентифікатором конкретного тренера;

- отримання списку всіх клієнтів, які беруть участь у поточному тренуванні;

4) управління записами на заняття (Registrations):

створення запису клієнта на обране тренування з фіксацією дати реєстрації;

- отримання переліку всіх існуючих у системі записів;

- динамічне оновлення статусу реєстрації (наприклад, переведення з PENDING в ACTIVE або CANCELLED);

- видалення (скасування) запису;

- вибірка всіх реєстрацій для конкретного тренування;

5) управління типами тренувань (WorkoutTypes):

- додавання нових напрямків (наприклад, «Йога», «CrossFit», «Кроссфіт»);

- перегляд списку доступних типів занять та видалення застарілих категорій;

- динамічне прив'язування певного типу до обраного тренування та можливість його вилучення;

6) повнотекстовий пошук (Search) :

- глобальний текстовий пошук серед тренувань за збігом у назві або текстовому описі;

- швидкий пошук тренерів за ПІБ чи спеціалізацією;

- пошук клієнтів за іменем або контактними даними;

7) аналітика та моніторинг (Analytics та Actuator):

- підрахунок загальної кількості створених тренувань та клієнтів у базі;

- агрегація кількості активних записів зі статусом ACTIVE;

- групування та підрахунок занять за їхніми категоріями/типами у вигляді мапи даних;

- динамічне сортування тренерів за рівнем популярності (кількістю занять);

- експортування технічних метрик системи, перевірка працездатності БД та підключення (Actuator endpoints).

Формування запитів, на які має давати відповідь програмна система.

1. Яка загальна кількість тренувань запланована в системі?

2. Які клієнти мають активний статус запису на тренування?

3. Які тренування закріплені за тренером із конкретним ID?

4. Скільки занять припадає на тип тренування «Силове тренування»?

5. Які тренери є найбільш затребуваними серед клієнтів?

6. Чи існують у системі клієнти або тренери з дублюючими адресами електронної пошти?

7. Які учасники записані на тренування, що відбудеться у визначений день?

8. Який поточний стан працездатності підключення додатка до бази даних PostgreSQL?

9. Чи містить опис або назва тренування ключове слово, передане користувачем у пошуковому параметрі?

10. Які деталі помилки виникли під час передачі клієнтом некоректного формату телефону?

Перелік звітів, які має формувати система:

1. Звіт про завантаженість тренерського складу: загальний перелік тренерів із зазначенням кількості проведених та запланованих занять для кожного.
2. Аналітичний звіт за типами занять: розподіл тренувань за категоріями для визначення пріоритетних напрямків розвитку клубу.
3. Звіт про активність клієнтської бази: кількісні показники зареєстрованих користувачів та обсяг їхніх активних записів.
4. Звіт про технічний стан сервісу: вивантаження системних метрик, використання пам'яті додатком та статусів підсистем для моніторингу працездатності.

Завдання автоматизації, що має реалізовувати система (Бізнес-правила).

Автоматизація та суворий контроль бізнес-логіки на рівні сервісного шару додатку дозволяють уникнути хаосу в даних, оптимізувати роботу персоналу та захистити систему від логічних помилок.

Основні автоматизовані завдання:

1. Контроль унікальності ідентифікаторів користувачів: сервіс автоматично блокує реєстрацію нового тренера або клієнта, якщо вказаний ними email вже зайнятий у системі, генеруючи виняток `DuplicateResourceException`.
2. Перевірка існування зв'язків: при створенні тренування чи реєстрації система автоматично перевіряє наявність переданих `trainerId` чи `memberId` у базі даних. Якщо зв'язана сутність не знайдена, операція переривається винятком `ResourceNotFoundException`.
3. Захист від дублювання активних операцій: сервіс блокує повторний запис одного й того самого клієнта на одне й те саме тренування, якщо у нього вже є активний статус запису (`ConflictOperationException`).
4. Захист цілісності при видаленні: система забороняє видаляти сутності (наприклад, тренера чи клієнта), якщо з ними в базі даних пов'язані чинні активні записи тренувань або реєстрацій, викидаючи `ConflictOperationException`.
5. Автоматична валідація форматів і значень: Bean Validation на вхідному рівні автоматично відсікає пусті рядки (перевірка `@NotBlank`), контролює межі

довжини тексту, позитивні значення тривалості занять, відповідність номерів телефонів та валідність структурування полів типу Enum для статусів. При порушенні правил автоматично формується статус 400 Bad Request із деталізацією помилкових полів у mapі errors.

Висновки до розділу 1

У першому розділі було детально проаналізовано предметну область діяльності сучасного спортивного клубу та визначено ключові проблеми, пов'язані з обробкою великих обсягів різномірних даних про розклад, клієнтів та персонал. На основі детального опису основних категорій користувачів системи (Адміністратора, Клієнта, Керівника) та їхніх історій використання (User Stories) сформовано повні вимоги до функціональності, валідації та аналітичних можливостей майбутнього backend-сервісу. Детальний аналіз функцій, бізнес-правил та автоматизованих завдань дозволив сформулювати чітку постановку задачі, яка стане основою для подальшого проектування архітектури бази даних, DTO-класів та програмної реалізації REST API.

2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

У цьому розділі наведено детальне проєктування внутрішньої структури даних, обґрунтування та опис багаторівневої архітектури серверної частини, а також повну специфікацію розробленого REST API інтерфейсу із прикладами структури запитів та відповідей.

2.1 Проєктування структури даних

Для забезпечення надійного збереження інформації та підтримки цілісності даних у системі було спроектовано реляційну структуру даних, яка відображає основні сутності бізнес-логіки фітнес-центру та зв'язки між ними.

Опис сутностей та атрибутивного складу.

Взаємодія об'єктів у базі даних координується за допомогою п'яти ключових сутностей:

1. Trainer (Тренер) — містить інформацію про викладацький склад клубу. Атрибути: id (первинний ключ, Long), firstName (String), lastName (String), specialization (String), email (унікальне поле, String).

2. Member (Клієнт клубу) — відображає дані відвідувачів. Атрибути: id (первинний ключ, Long), firstName (String), lastName (String), email (унікальне поле, String), phone (String).

3. Workout (Тренування) — описує конкретне групове чи індивідуальне заняття у розкладі. Атрибути: id (первинний ключ, Long), title (String), description (Text), date (LocalDateTime), durationMinutes (Integer).

4. WorkType (Тип тренування) — категоризує напрямки фітнесу (наприклад: CrossFit, Йога). Атрибути: id (первинний ключ, Long), name (String), description (String).

5. Registration (Запис на тренування) — сутність-реєстрація, яка зв'язує відвідувача та заняття. Атрибути: id (первинний ключ, Long), registrationDate (LocalDateTime), status (перерахування Enum: ACTIVE, CANCELLED).

Специфікація зв'язків між сутностями:

- Trainer — Workout (1 : N): один тренер може проводити безліч тренувань, але кожне конкретне тренування закріплюється за одним відповідальним тренером (@ManyToOne).
- Workout — WorkType (M : N): одне тренування може поєднувати кілька напрямків або типів навантажень, і водночас один тип тренування може відноситися до багатьох занять у розкладі. Реалізується через проміжну таблицю мапінгу зв'язків.
- Member — Workout (M : N): багато клієнтів можуть відвідувати багато тренувань. На рівні бізнес-логіки цей зв'язок деталізується за допомогою проміжної сутності реєстрації.
- Workout — Registration (1 : N) та Member — Registration (1 : N): сутність Registration виступає власником зв'язків та містить зовнішні ключі до таблиць клієнта та конкретного заняття, що дозволяє гнучко керувати статусом запису на рівні окремої транзакції.

2.2 Опис архітектури застосунку

Backend-сервіс управління спортивним клубом побудовано на базі фреймворку Spring Boot 3 із використанням класичного трирівневого архітектурного шаблону (Layered Architecture), що забезпечує чітке розділення відповідальності (Separation of Concerns), полегшує тестування системи та ізолює бізнес-правила від протоколів передачі даних.

Характеристика архітектурних рівнів:

1. Рівень представлення (Controller Layer / REST API): представлений класами з анотацією @RestController (MemberController, TrainerController, WorkoutController тощо). Цей шар відповідає виключно за прийом HTTP-запитів, десеріалізацію JSON у вхідні об'єкти запиту, запуск декларативної валідації (@Valid) та повернення стандартизованих HTTP-відповідей. Контролери проекту є "тонкими" — вони не містять бізнес-логіки та не взаємодіють із JPA-сутностями

напрямку, використовуючи виключно класи DTO (Data Transfer Object) для обміну даними із зовнішнім світом.

2. Рівень бізнес-логіки (Service Layer): представлений класами з анотацією @Service (MemberService, RegistrationService, TrainerService, WorkoutService). Шар сервісів інкапсулює ключові правила предметної області. Тут виконується перевірка логічної коректності операцій, контроль унікальності email-адрес перед збереженням, валідація наявності пов'язаних ID у базі, а також генерація кастомних винятків (DuplicateResourceException, ResourceNotFoundException). Сервіси отримують дані від контролерів у вигляді моделей або параметрів, координують транзакції та взаємодіють із шаром доступу до даних.

3. Рівень доступу до даних (Repository Layer / Persistence Layer): представлений інтерфейсами, які розширюють JpaRepository зі складу фреймворку Spring Data JPA. Цей рівень забезпечує абстракцію над базою даних (в даному проекті конфігурація адаптована під роботу з СУБД PostgreSQL/Oracle). За допомогою технології Hibernate об'єктно-реляційний мапінг (ORM) трансформує операції над Java-об'єктами сутностей у нативні та оптимізовані SQL-запити.

Залежності між компонентами.

У системі реалізовано принцип інверсії управління (IoC) за допомогою впровадження залежностей через конструктор (Constructor Injection), що гарантує незмінність залежностей та спрощує модульне тестування. Потoki даних рухаються строго зверху вниз:

– HTTP Request → Controller → Service → Repository → Database.

Зворотний рух даних супроводжується мапінгом (трансформацією) Entity-класів у незмінні структури відповіді (Java Records):

– Entity (DB State) → Service → DTO (Response) → Controller → JSON HTTP Response.

2.3 Опис REST API

Веб-інтерфейс розробленого сервісу спроектовано відповідно до архітектурних обмежень стилю REST. Для документування та візуалізації API інтегровано специфікацію OpenAPI 3 (Swagger) за допомогою конфігураційного класу `OpenApiConfig`.

Перелік реалізованих ендпоінтів та їх призначення.

У системі розгорнуто та підтримується 6 функціональних груп ендпоінтів:

1) клієнти клубу (`/members`):

- `POST /members` — реєстрація нового відвідувача клубу із застосуванням правил валідації (вхід: `MemberCreateRequest`, вихід: `MemberResponse`);
- `GET /members` — отримання повного переліку зареєстрованих клієнтів (вихід: `List<MemberResponse>`);
- `GET /members/{id}` — отримання картки профілю конкретного клієнта за його ідентифікатором;
- `PUT /members/{id}` — модифікація персональних або контактних даних існуючого клієнта (вхід: `MemberUpdateRequest`);
- `DELETE /members/{id}` — видалення клієнта з бази даних (повертає статус 204 No Content);
- `GET /members/{id}/workouts` — отримання індивідуального розкладу занять (тренувань), на які записаний відвідувач;

2) тренери (`/trainers`):

- `POST /trainers` — додавання нового тренера із зазначенням його спеціалізації та email;
- `GET /trainers` — отримання всього списку викладацького складу;
- `GET /trainers/{id}` — пошук тренера за унікальним ID;
- `PUT /trainers/{id}` — оновлення інформації про тренера (вхід: `TrainerUpdateRequest`);
- `DELETE /trainers/{id}` — видалення картки тренера із системи;
- `GET /trainers/{id}/workouts` — вивантаження розкладу тренувань, які проводить цей тренер;

3) тренування (/workouts):

- POST /workouts — створення нової позиції тренування в сітці розкладу (вхід: WorkoutCreateRequest);
- GET /workouts — отримання повного списку занять фітнес-центру;
- GET /workouts/{id} — отримання розгорнутих деталей конкретного тренування;
- PUT /workouts/{id} — оновлення параметрів та часу проведення існуючого тренування;
- DELETE /workouts/{id} — вилучення тренування з загального розкладу;
- GET /workouts/trainer/{trainerId} — вибірка всіх занять, що закріплені за певним тренером;
- GET /workouts/{id}/members — отримання списку всіх відвідувачів, які є учасниками конкретного заняття;
- POST /workouts/{id}/members/{memberId} — пряме адміністративне додавання клієнта на заняття в розкладі;

4) записи на тренування (/registrations):

- POST /registrations — оформлення та створення нової клієнтської заявки-запису на тренування;
- GET /registrations — перегляд журналу всіх створених у базі даних записів;
- GET /registrations/{id} — пошук та перегляд окремої заявки за її ідентифікатором;
- PUT /registrations/{id}/status — динамічна зміна статусу запису (вхід: RegistrationStatusUpdateRequest, приймає ACTIVE або CANCELLED);
- DELETE /registrations/{id} — видалення запису (скасування заявки на тренування з бази даних);
- GET /workouts/{id}/registrations — отримання списку офіційних заявок (реєстрацій) на конкретне тренування;

5) типи тренувань (/workout-types):

- POST /workout-types — реєстрація нового фітнес-напрямку чи спортивної дисципліни;
- GET /workout-types — повернення повного переліку існуючих категорій занять;
- GET /workout-types/{id} — отримання детальної інформації про тип за його ID;
- DELETE /workout-types/{id} — повне видалення категорії або напрямку з системи;
- POST /workouts/{id}/types/{typeId} — призначення та прив'язка категорії фітнесу до конкретного тренування;
- DELETE /workouts/{id}/types/{typeId} — вилучення категорії зі структури обраного заняття;

6) повнотекстовий пошук (/search):

- GET /search/workouts?query= — пошук занять за збігом символів у назві або детальному текстовому описі;
- GET /search/trainers?query= — фільтрація та пошук тренерів за ім'ям, прізвищем або спеціалізацією;
- GET /search/members?query= — швидкий пошук клієнтів за шаблоном ПІБ чи адресою електронної пошти;

7) аналітичні запити (/analytics):

- GET /analytics/workouts/count — отримання загальної кількості запланованих тренувань у системі;
- GET /analytics/members/count — повернення сумарного числа зареєстрованих відвідувачів клубу;
- GET /analytics/workouts/active — підрахунок кількості чинних реєстрацій клієнтів зі статусом активності;
- GET /analytics/workouts/by-type — отримання кількості проведених тренувань, розподілених за типами;
- GET /analytics/trainers/popular — виведення списку найпопулярніших тренерів на основі кількості занять;

8) метрики сервісу (/actuator):

- GET /actuator/health — службова перевірка стану працездатності (Health Check) веб-сервісу;
- GET /actuator/metrics — отримання повного списку доступних технічних метрик застосунку;
- GET /actuator/prometheus — експорт зібраних метрик у форматі для систем моніторингу Prometheus.

Висновки до розділу 2

У ході проектування програмного забезпечення було повністю сформовано реляційну структуру даних, визначено сутності та типи зв'язків між ними, що виключає аномалії та дублювання інформації. Впровадження трирівневої архітектури (Controller–Service–Repository) на базі Spring Boot 3 дозволило відокремити бізнес-правила спортивного клубу від логіки обробки веб-запитів. Опис REST API інтерфейсу, підкріплений автоматичною генерацією OpenAPI/Swagger, та розробка системи типізованих об'єктів DTO гарантують безпеку даних, сувору вхідну валідацію та прозору систему інформування про клієнтські або системні помилки.

3 РЕАЛІЗАЦІЯ ПРОГРАМНОГО ПРОДУКТУ

Цей розділ присвячено опису практичної реалізації компонентів інформаційної системи фітнес-клубу. Застосунок побудовано на базі архітектурного шаблону Layered Architecture (Багатошарова архітектура), що дозволяє відокремити конфігурацію, представлення даних, бізнес-логіку та взаємодію з базою даних.

3.1 Конфігурація системи (Configuration Layer)

Конфігураційний шар відповідає за встановлення з'єднання з базою даних, налаштування інструментів моніторингу та документації API.

Основні параметри підключення до СКБД, логування та активації метрик винесені у файл `application.properties`:

```
// Налаштування підключення до бази даних PostgreSQL
spring.datasource.url=jdbc:postgresql://localhost:5432/sportclub
spring.datasource.username=postgres
spring.datasource.password=2006
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=create-drop

//Активация та налаштування Spring Boot Actuator для збору метрик
management.endpoints.web.exposure.include=health,metrics,prometheus
management.endpoint.health.show-details=always
```

Для інтеграції специфікації OpenAPI 3 (Swagger) створено конфігураційний клас `OpenApiConfig`, який реєструє інформацію про версію та призначення розробленого REST API:

```
package ua.opnu.labwork2.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Contact;
import io.swagger.v3.oas.models.info.Info;
import org.springframework.context.annotation.Bean;
```



```

import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI customOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Backend-сервіс управління спортивним клубом")
                .version("1.0.0")
                .description("REST API для автоматизації обліку тренерів, клієнтів, " +
                    "тренувань та записів на заняття у фітнес-центрах. " +
                    "Реалізовано на базі Spring Boot 3, Spring Data JPA та Oracle DB.")
                .contact(new Contact()
                    .name("Кафедра ІКС, ОНПУ")
                    .email("student.opnu@gmail.com")));
    }
}

```

3.2 Моделі даних (Entity-класи та DTO)

Шар моделей складається з двох типів компонентів:

1. Entity-класи — відображають структуру таблиць бази даних та керуються специфікацією JPA (Hibernate).

2. DTO (Data Transfer Objects) — реалізовані через сучасні Java Records. Вони забезпечують безпечний обмін даними між сервером та клієнтом, а також реалізують правила валідації вхідних полів (Bean Validation).

Фрагмент реалізації сутності Workout (Entity):

```

package ua.opnu.labwork2.model;

import jakarta.persistence.*;

import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "workouts")

```

```

public class Workout {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    @Column(columnDefinition = "TEXT")
    private String description;

    private LocalDateTime date;

    private Integer durationMinutes;

    @ManyToOne
    @JoinColumn(name = "trainer_id")
    private Trainer trainer;

    @ManyToMany
    @JoinTable(
        name = "workout_members",
        joinColumns = @JoinColumn(name = "workout_id"),
        inverseJoinColumns = @JoinColumn(name = "member_id")
    )
    private final Set<Member> members = new HashSet<>();

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "workout_types_mapping",
        joinColumns = @JoinColumn(name = "workout_id"),
        inverseJoinColumns = @JoinColumn(name = "type_id")
    )
    private final Set<WorkType> workoutTypes = new HashSet<>();

    public Workout() {
    }

    public void addMember(Member member) {
        this.members.add(member);
        member.getWorkouts().add(this);
    }
    // Геттери, сеттери
}

```

Фрагмент реалізації об'єктів перенесення даних (DTO).

Для отримання очищених даних клієнта використовується не сама сутність, а незмінний об'єкт `MemberResponse`. Для перевірки вхідних даних при реєстрації створено `MemberCreateRequest`:

```

package ua.opnu.labwork2.dto;

import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;

// DTO-запит із правилами валідації полів
public record MemberCreateRequest(
    @Schema(description = "Ім'я клієнта клубу", example =
"Олександр", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "First name is required and cannot be
empty or blank")
    @Size(min = 2, max = 50, message = "First name must be
between 2 and 50 characters")
    String firstName,

    @Schema(description = "Прізвище клієнта клубу", example =
"Коваленко", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Last name is required and cannot be
empty or blank")
    @Size(min = 2, max = 50, message = "Last name must be
between 2 and 50 characters")
    String lastName,

    @Schema(description = "Унікальна електронна пошта клієнта",
example = "olex.koval@gmail.com", requiredMode =
Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Email is required")
    @Email(message = "Email must match valid email format")
    @Size(max = 100, message = "Email cannot exceed 100
characters")
    String email,

    @Schema(description = "Контактний номер телефону", example =
"+380501234567", requiredMode = Schema.RequiredMode.REQUIRED)
    @NotBlank(message = "Phone number is required")
    @Size(min = 10, max = 20, message = "Phone must be between
10 and 20 characters")
    @Pattern(regexp = "[0-9+\\s]+$", message = "Phone can only
contain digits, spaces, and '+' symbol")
    String phone
) {}

```

3.3 Репозиторії (Repository Layer)

Шар репозиторіїв представлений інтерфейсами, які наслідують JpaRepository. Вони забезпечують виконання базових CRUD-операцій, а також містять кастомні методи пошуку та JPQL/SQL запити для формування аналітики.

Реалізація репозиторію Workout:

```
package ua.opnu.labwork2.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import ua.opnu.labwork2.model.Workout;

import java.util.List;

@Repository
public interface WorkoutRepository extends JpaRepository<Workout, Long> {
    List<Workout> findByTrainerId(Long trainerId);
    List<Workout> findByMembers_Id(Long memberId);
}
```

3.4 Сервіси (Service Layer)

У шарі сервісів зосереджена вся бізнес-логіка застосунку. Сервіси викликають методи репозиторіїв, здійснюють мапінг між Entity та DTO за допомогою методів-конвертерів, а також генерують винятки (наприклад, ResourceNotFoundException), якщо запитувані об'єкти відсутні в базі.

Реалізація сервісу Workout:

```
package ua.opnu.labwork2.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import ua.opnu.labwork2.exception.BadRequestException;
import ua.opnu.labwork2.exception.ConflictOperationException;
import ua.opnu.labwork2.exception.ResourceNotFoundException;
import ua.opnu.labwork2.model.Member;
import ua.opnu.labwork2.model.RegistrationStatus;
import ua.opnu.labwork2.model.WorkType;
import ua.opnu.labwork2.model.Workout;
import ua.opnu.labwork2.repository.RegistrationRepository;
import ua.opnu.labwork2.repository.WorkoutRepository;
import ua.opnu.labwork2.repository.WorkoutTypeRepository;

import java.util.List;
```

```

@Service
public class WorkoutService {

    private final WorkoutRepository workoutRepository;
    private final WorkoutTypeRepository typeRepository;
    private final MemberService memberService;
    private final RegistrationRepository registrationRepository; //
Додано для контролю записів

    public WorkoutService(WorkoutRepository wr,
WorkoutTypeRepository tr, MemberService ms, RegistrationRepository
rr) {
        this.workoutRepository = wr;
        this.typeRepository = tr;
        this.memberService = ms;
        this.registrationRepository = rr;
    }

    @Transactional
    public Workout create(Workout workout) {
        if (workout.getDurationMinutes() <= 0 ||
workout.getDurationMinutes() > 480) {
            throw new BadRequestException("Workout duration must be
between 1 and 480 minutes.");
        }
        return workoutRepository.save(workout);
    }

    public List<Workout> getAll() { return
workoutRepository.findAll(); }

    public Workout getById(Long id) {
        return workoutRepository.findById(id)
            .orElseThrow(() -> new
ResourceNotFoundException("Workout not found with id: " + id));
    }

    public List<Workout> getByTrainerId(Long trainerId) {
        return workoutRepository.findByTrainerId(trainerId);
    }

    public List<Workout> getByMemberId(Long memberId) {
        return workoutRepository.findByMembers_Id(memberId);
    }

    @Transactional
    public void addMemberToWorkout(Long workoutId, Long memberId) {
        Workout workout = getById(workoutId);
        Member member = memberService.getById(memberId);
        workout.addMember(member);
        workoutRepository.save(workout);
    }
}

```

```

@Transactional
public void addTypeToWorkout(Long workoutId, Long typeId) {
    Workout workout = getById(workoutId);
    WorkType type = typeRepository.findById(typeId)
        .orElseThrow(() -> new
ResourceNotFoundException("Type not found with id: " + typeId));
    workout.getWorkoutTypes().add(type);
    workoutRepository.save(workout);
}

@Transactional
public void removeTypeFromWorkout(Long workoutId, Long typeId) {
    Workout workout = getById(workoutId);
    workout.getWorkoutTypes().removeIf(t ->
t.getId().equals(typeId));
    workoutRepository.save(workout);
}

@Transactional
public void delete(Long id) {
    if (!workoutRepository.existsById(id)) {
        throw new ResourceNotFoundException("Workout not found
with id: " + id);
    }

    boolean hasActiveRegistrations =
registrationRepository.existsActiveByWorkout(id,
RegistrationStatus.ACTIVE);
    if (hasActiveRegistrations) {
        throw new ConflictOperationException("Cannot delete
workout: It has active member registrations.");
    }

    workoutRepository.deleteById(id);
}
}

```

3.5 Контролери (Controller Layer)

Контролери відповідають за обробку вхідних HTTP-запитів від клієнта, мапінг URL-шляхів, повернення відповідних HTTP-статусів та взаємодію із сервісним шаром застосунку.

Реалізація контролеру Workout:

```
package ua.opnu.labwork2.controller;
```

```

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import ua.opnu.labwork2.dto.MemberResponse;
import ua.opnu.labwork2.dto.RegistrationResponse;
import ua.opnu.labwork2.dto.WorkoutCreateRequest;
import ua.opnu.labwork2.dto.WorkoutResponse;
import ua.opnu.labwork2.model.Workout;
import ua.opnu.labwork2.service.RegistrationService;
import ua.opnu.labwork2.service.WorkoutService;

import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/workouts")
@Tag(name = "Тренування", description = "Управління розкладом
занять, закріпленням тренерів та типами тренувань")
public class WorkoutController {

    private final WorkoutService workoutService;
    private final RegistrationService registrationService;

    public WorkoutController(WorkoutService ws, RegistrationService
rs) {
        this.workoutService = ws;
        this.registrationService = rs;
    }

    @PostMapping
    @Operation(summary = "Створити нове тренування")
    public ResponseEntity<WorkoutResponse> create(@Valid
@RequestBody WorkoutCreateRequest request) {
        Workout workout = new Workout();
        workout.setTitle(request.title());
        workout.setDescription(request.description());
        workout.setDate(request.date());
        workout.setDurationMinutes(request.durationMinutes());

        if (request.trainerId() != null) {
            ua.opnu.labwork2.model.Trainer trainer = new
ua.opnu.labwork2.model.Trainer();
            trainer.setId(request.trainerId());
            workout.setTrainer(trainer);
        }

        Workout saved = workoutService.create(workout);

```

```

        return
        ResponseEntity.status(HttpStatus.CREATED).body(WorkoutResponse.fromEntity(saved));
    }

    @GetMapping
    @Operation(summary = "Отримати всі тренування")
    public ResponseEntity<List<WorkoutResponse>> getAll() {
        List<WorkoutResponse> responseList =
workoutService.getAll().stream()
            .map(WorkoutResponse::fromEntity)
            .toList();
        return ResponseEntity.ok(responseList);
    }

    @GetMapping("/{id}")
    @Operation(summary = "Отримати деталі тренування за ID")
    public ResponseEntity<WorkoutResponse> getById(@PathVariable Long id) {
        Workout workout = workoutService.getById(id);
        return
        ResponseEntity.ok(WorkoutResponse.fromEntity(workout));
    }

    @PostMapping("/{id}/members/{memberId}")
    @Operation(summary = "Додати клієнта на тренування напряму")
    public ResponseEntity<Void> addMember(@PathVariable Long id,
@PathVariable Long memberId) {
        workoutService.addMemberToWorkout(id, memberId);
        return ResponseEntity.ok().build();
    }

    @GetMapping("/trainer/{trainerId}")
    @Operation(summary = "Пошук тренувань конкретного тренера")
    public ResponseEntity<List<WorkoutResponse>>
getByTrainer(@PathVariable Long trainerId) {
        List<WorkoutResponse> responseList =
workoutService.getByTrainerId(trainerId).stream()
            .map(WorkoutResponse::fromEntity)
            .toList();
        return ResponseEntity.ok(responseList);
    }

    @GetMapping("/{id}/members")
    @Operation(summary = "Отримати список учасників заняття")
    public ResponseEntity<Set<MemberResponse>>
getMembers(@PathVariable Long id) {
        Workout workout = workoutService.getById(id);
        Set<MemberResponse> responseSet =
workout.getMembers().stream()
            .map(MemberResponse::fromEntity)
            .collect(Collectors.toSet());
        return ResponseEntity.ok(responseSet);
    }

```



```

    }

    @GetMapping("/{id}/registrations")
    @Operation(summary = "Переглянути записи на тренування")
    public ResponseEntity<List<RegistrationResponse>>
getRegistrations(@PathVariable Long id) {
        List<RegistrationResponse> responseList =
registrationService.getByWorkoutId(id).stream()
            .map(RegistrationResponse::fromEntity)
            .toList();
        return ResponseEntity.ok(responseList);
    }

    @PostMapping("/{id}/types/{typeId}")
    @Operation(summary = "Призначити тип/напрямок для тренування")
    public ResponseEntity<Void> addType(@PathVariable Long id,
@PathVariable Long typeId) {
        workoutService.addTypeToWorkout(id, typeId);
        return ResponseEntity.ok().build();
    }

    @DeleteMapping("/{id}/types/{typeId}")
    @Operation(summary = "Видалити тип тренування")
    public ResponseEntity<Void> removeType(@PathVariable Long id,
@PathVariable Long typeId) {
        workoutService.removeTypeFromWorkout(id, typeId);
        return ResponseEntity.noContent().build();
    }
}

```

3.6 Глобальна обробка виняткових ситуацій (Exception Handling)

Для забезпечення стабільної структури відповідей сервера під час помилок, реалізовано клас `GlobalExceptionHandler`. Він перехоплює помилки валідації (`MethodArgumentNotValidException`) та відсутності ресурсів, повертаючи клієнту об'єкт `ApiErrorResponse` замість стандартного системного `StackTrace`.

Реалізація `GlobalExceptionHandler`:

```

package ua.opnu.labwork2.exception;

import io.swagger.v3.oas.annotations.Hidden;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

```

```

import
org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.method.annotation.MethodArgumentTypeMismatch
Exception;
import ua.opnu.labwork2.dto.ApiErrorResponse;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.NoSuchElementException;

@RestControllerAdvice
@Hidden
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse>
handleValidationExceptions(
        MethodArgumentNotValidException ex,
        WebRequest request
    ) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach((error) -> {
            String fieldName = ((FieldError) error).getField();
            String errorMessage = error.getDefaultMessage();
            errors.put(fieldName, errorMessage);
        });

        String path = request.getDescription(false).replace("uri=",
"" );

        ApiErrorResponse errorResponse = new ApiErrorResponse(
            LocalDateTime.now(),
            HttpStatus.BAD_REQUEST.value(),
            HttpStatus.BAD_REQUEST.getReasonPhrase(),
            "Validation failed for one or more fields",
            path,
            errors
        );

        return
ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errorResponse);
    }

    @ExceptionHandler({
        MethodArgumentTypeMismatchException.class,
        HttpMessageNotReadableException.class,

```

```

        BadRequestException.class
    })
    public ResponseEntity<ApiErrorResponse> handleBadRequestGroup(
        Exception ex,
        WebRequest request
    ) {
        String customMessage = "The request contains invalid data or
malformed structure";
        if (ex instanceof BadRequestException) {
            customMessage = ex.getMessage();
        } else if (ex instanceof
MethodArgumentTypeMismatchException) {
            customMessage = "Parameter type mismatch: " +
ex.getMessage();
        }

        return createErrorResponse(HttpStatus.BAD_REQUEST,
customMessage, request);
    }

    @ExceptionHandler({
        ResourceNotFoundException.class,
        NoSuchElementException.class
    })
    public ResponseEntity<ApiErrorResponse> handleNotFoundGroup(
        Exception ex,
        WebRequest request
    ) {
        return createErrorResponse(HttpStatus.NOT_FOUND,
ex.getMessage(), request);
    }

    @ExceptionHandler({
        DuplicateResourceException.class,
        ConflictOperationException.class,
        DataIntegrityViolationException.class
    })
    public ResponseEntity<ApiErrorResponse> handleConflictGroup(
        Exception ex,
        WebRequest request
    ) {
        String message = ex.getMessage();
        if (ex instanceof DataIntegrityViolationException) {
            message = "Database integrity constraint violation:
Duplicate key or foreign key conflict.";
        }
        return createErrorResponse(HttpStatus.CONFLICT, message,
request);
    }

    @ExceptionHandler({
        DatabaseOperationException.class,
        RuntimeException.class,

```

```

        Exception.class
    })
    public ResponseEntity<ApiErrorResponse>
handleInternalServerErrorGroup(
        Exception ex,
        WebRequest request
    ) {
        return createErrorResponse(HttpStatus.INTERNAL_SERVER_ERROR,
ex.getMessage(), request);
    }

    private ResponseEntity<ApiErrorResponse> createErrorResponse(
        HttpStatus status,
        String message,
        WebRequest request
    ) {
        String path = request.getDescription(false).replace("uri=",
"");

        ApiErrorResponse errorResponse = new ApiErrorResponse(
            LocalDateTime.now(),
            status.value(),
            status.getReasonPhrase(),
            message,
            path,
            null // Для звичайних винятків (не валідації полів)
мапа errors дорівнює null
        );

        return ResponseEntity.status(status).body(errorResponse);
    }
}

```

Висновки до розділу 3

У ході виконання практичного етапу роботи на базі багатоварової архітектури Spring Boot було успішно реалізовано веб-сервіс для автоматизації процесів обліку та управління фітнес-клубом. Розробка стійкого шару доступу до даних за допомогою JPA (Hibernate) та впровадження незмінних об'єктів перенесення даних (DTO Records) дозволили оптимізувати аналітичні вибірки та гарантувати високий рівень безпеки й валідації даних. Крім того, інтеграція підсистеми Spring Boot Actuator та специфікації OpenAPI 3 забезпечила ефективний моніторинг технічних метрик застосунку й повну автоматизацію документування створених REST-ендпоінтів.

4 Тестування та налагодження

У цьому розділі описано методику та процес перевірки працездатності розробленої системи, наведено інструменти тестування, результати верифікації основних функцій, а також аналіз виявлених помилок та способів їх усунення.

4.1 Інструменти тестування та налагодження

Swagger UI (/swagger-ui/index.html): інтегрований у Spring Boot застосунок за допомогою бібліотеки springdoc-openapi-starter-webmvc-ui. Він автоматично аналізує контролери, зчитує метадані анотацій та формує інтерактивну документацію. Використовувався для швидкої перевірки форматів запитів/відповідей, схем валідації та генерації тестових cURL-команд.

Postman: застосовувався для побудови комплексних тестових сценаріїв, автоматичного збереження колекцій запитів, перевірки крайових умов, симуляції помилок валідації заголовків та передачі некоректних типів даних у тілі HTTP-запиту. Усі запити параметризовані за допомогою змінних оточення (наприклад, {{base_url}}), що дозволяє миттєво перемикає тестування з локального сервера (localhost:8080) на віддалений хостинг (Render.com).

4.2 Структура тестової колекції в Postman

Для автоматизації процесу перевірки працездатності розробленого REST API та проведення регресійного тестування було спроектовано та наповнено однойменну колекцію запитів у середовищі Postman. Усі ендпоінти чітко структуровані за логічними папками (модулями системи), що відповідають основним доменним моделям фітнес-клубу.

Ієрархічна архітектура розробленої Postman-колекції із зазначенням HTTP-методів та призначення кожного запиту:

- 1) модуль «Trainers» (управління тренерами):

- POST Create Trainer — створення нового профілю тренера (перевірка унікальності email);
- GET Get All Trainers — отримання повного списку інструкторів фітнес-клубу;
- GET Get Trainer By ID — вибірка конкретного тренера за його унікальним ідентифікатором;
- PUT Update Trainer — оновлення анкетних даних, кваліфікації або контактів тренера;
- DELETE Delete Trainer — каскадне або безпечне видалення картки тренера із системи;
- GET GetTrainers Workout — отримання переліку занять (розкладу), закріплених за конкретним тренером;

2) модуль «Members» (Управління відвідувачами):

- POST Create Member — реєстрація нового клієнта в системі (валідація телефону, імені, email);
- GET Get All Members — вивантаження списку всіх зареєстрованих відвідувачів;
- GET Get Member By ID — перегляд картки профілю клієнта за ID;
- PUT Update Member — редагування поточної контактної інформації користувача;
- DELETE Delete Member — видалення профілю з пре-валідацією відсутності активних записів;
- GET GetMembersWorkout — отримання списку тренувань, які відвідував або планує відвідати клієнт;

3) модуль «Registrations» (Оформлення та облік заявок):

- POST Create Registration — створення нового запису клієнта на тренування (валідація обмежень «із майбутнього» та минулих занять);
- GET Get All Registrations — аудит усіх створених у системі заявок;
- GET Get Registration By ID — перевірка деталей конкретного запису за ID;

- PUT Update registrations — модифікація запису (наприклад, зміна статусу або пов'язаних параметрів);
- DELETE Delete Registration — видалення реєстраційного документа з бази даних;
- GET GetWorkoutRegistrations — отримання списку всіх записів, оформлених на конкретне тренування;

4) модуль «Workouts» (Розклад та координація тренувань):

- POST Create Workout — додавання нового сеансу тренування (перевірка тривалості сесії в межах 1–480 хв);
- GET Get All Workouts — отримання повної сітки занять фітнес-клубу;
- GET Get Workout By ID — запит детальної інформації про заняття за ID;
- PUT Update Workout — коригування часу, опису чи назви тренування;
- DELETE Delete Workout — безпечне видалення заняття (блокується за наявності активних клієнтів);
- GET GetWorkouts Members — перегляд списку відвідувачів, які зафіксовані на цьому занятті;
- GET GetWorkouts Trainers — отримання інформації про тренера, який проводить це заняття;

5) модуль «Workout Types» (Класифікація та категорії занять):

- POST Create WorkoutType — створення нової категорії занять;
- GET Get All Workout Types — виведення повного списку доступних у клубі напрямків;
- GET Get Workout Type By ID — перегляд опису конкретного типу за ID;
- DELETE Delete Workout Type — видалення категорії з довідника;
- POST AddTypeToWorkouts — динамічне присвоєння категорії конкретному тренуванню (додавання зв'язку в @ManyToMany таблицю);

– DELETE DeleteTypeFromWorkout — вилучення категорії зі структури тренування.

4.3 Результати тестування основних функцій системи

Нижче наведено практичні приклади виконання HTTP-запитів у середовищі Postman, які демонструють успішні сценарії роботи системи та її поведінку при виникненні помилок.

Приклад 1. Успішне створення запису на тренування.

Цей тест перевіряє базовий сценарій реєстрації користувача на групове заняття. Запит ініціює перевірки наявності сутностей, контроль дат і транзакційне збереження даних.

HTTP-метод: POST.

URL: `http://localhost:8080/registrations`.

Тіло запиту (Request Body):

```
{
  "memberId": 1,
  "workoutId": 1,
  "status": "ACTIVE",
  "registrationDate": "2026-05-27T18:00:00"
}
```

Код відповіді сервера: 201 Created.

Тіло відповіді (Response Body):

```
{
  "id": 1,
  "registrationDate": "2026-05-27T18:00:00",
  "status": "ACTIVE"
}
```

Приклад 2. Помилка валідації вхідних даних (Validation Failure).

Цей тест демонструє поведінку системи у випадку надсилання клієнтом JSON-документа, структура якого порушує декларативні обмеження (Constraints), встановлені на рівні полів DTO-моделі.

HTTP-метод: POST.

URL: `http://localhost:8080/members`.

Тіло запиту (Некоректні дані):

```
{
  "firstName": "",
  "lastName": "Петренко",
  "email": "invalid-email-format",
  "phone": "abc123phone"
}
```

Код відповіді сервера: 400 Bad Request.

Тіло відповіді (Формат `ApiErrorResponse`):

```
{
  "timestamp": "2026-06-08T12:39:34.1710515",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for one or more fields",
  "path": "/members",
  "errors": {
    "firstName": "First name is required and cannot be empty or blank",
    "phone": "Phone can only contain digits, spaces, and '+' symbol",
    "email": "Email must match valid email format"
  }
}
```

Аналіз логіки обробки запиту:

1. Перехоплення на рівні десеріалізації: під час зчитування тіла запиту Спрінг викликає валідатор `Jakarta Bean Validation` для перевірки обмежень DTO-класу.

2. Генерація винятку: оскільки поле `firstName` виявилося порожнім (`@NotBlank`), у полі `email` відсутній символ `@` (`@Email`), а `phone` містить літери замість регулярного виразу (`@Pattern`), валідатор блокує передачу запиту до сервісного шару та викидає `MethodArgumentNotValidException`.

3. Централізована обробка: виняток перехоплюється методом `handleValidationExceptions` у `GlobalExceptionHandler`. Метод ітерується по списку помилок (`ex.getBindingResult().getAllErrors()`), дістає імена полів і тексти дефолтних повідомлень помилок, після чого пакує їх у мапу `errors`. В результаті клієнт отримує чистий JSON-об'єкт із HTTP-статусом 400 без витоку внутрішньої інформації про стек викликів (`stack trace`) сервера.

Приклад 3. Успішне оновлення даних клієнта з перевіркою на дублікати (`Idempotent PUT Path`).

Цей сценарій верифікує коректність модифікації профілю відвідувача. Система повинна дозволити збереження зміненого email, якщо він належить цьому ж користувачу або є абсолютно новим у системі, але заблокувати операцію, якщо такий email уже зайнятий іншою особою.

HTTP-метод: PUT.

URL: `http://localhost:8080/members/1`.

Тіло запиту (`Request Body`):

```
{
  "firstName": "Олександр",
  "lastName": "Коваленко",
  "email": "olex.new.email@gmail.com",
  "phone": "+380671112233"
}
```

Код відповіді сервера: 200 OK.

Тіло відповіді (`Response Body`):

```
{
  "id": 1,
  "firstName": "Олександр",
  "lastName": "Коваленко",
  "email": "olex.new.email@gmail.com",
  "phone": "+380671112233"
}
```

Аналіз логіки обробки запиту:

1. Пошук існуючої сутності: `MemberService.update(id, details)` спочатку звертається до бази даних за допомогою методу `getById(1)`. Якщо запис знайдено, він завантажується в пам'ять (`Persistence Context`).

2. Ізольована перевірка унікальності: сервіс викликає метод репозиторію `memberRepository.existsByEmailAndIdNot("olex.new.email@gmail.com", 1L)`. Цей SQL-запит перевіряє, чи є в таблиці інші користувачі (де `id != 1`), які вже використовують таку адресу. Оскільки email вільний, перевірка повертає `false`.

3. Мутація та синхронізація з БД: через сетери об'єкта `Member` оновлюються відповідні поля. При завершенні транзакції `Hibernate` автоматично генерує команду `UPDATE` до СКБД. Клієнту повертається вже модифікований об'єкт із кодом `200 OK`.

Висновки до розділу 4

За результатами функціонального тестування за допомогою інструментів `Postman` та `Swagger UI` повністю підтверджено працездатність усіх розроблених ендпоінтів та їхню готовність до інтеграції з клієнтською частиною застосунку. Налагоджена система глобальної обробки винятків та `Fail-Fast` валідації на сервісному шарі забезпечує стовідсотковий захист бази даних від збереження логічно суперечливих даних, дублікатів або порушення реляційної цілісності. Тестові сценарії довели, що архітектура ефективно ізолює бізнес-логіку від збоїв, повертаючи клієнту інформативні стандартизовані відповіді у форматі `ApiErrorResponse`.

ЗАГАЛЬНІ ВИСНОВКИ

У ході виконання курсової роботи було успішно спроектовано та реалізовано веб-орієнтований backend-сервіс та реляційну базу даних для автоматизації діяльності сучасного спортивного клубу. На основі аналізу предметної області та зібраних вимог користувачів (User Stories) було повністю ліквідовано проблеми фрагментарного зберігання інформації, мінімізовано ручну працю та виключено ризики логічних помилок при координації розкладу.

Основні результати роботи та ступінь досягнення мети:

- спроектовано та нормалізовано реляційну базу даних на базі СУБД PostgreSQL, що охоплює 5 ключових сутностей (Trainer, Member, Workout, Registration, WorkoutType) та надійно підтримує цілісність даних через обмеження первинних і зовнішніх ключів;

- розроблено REST API сервіс на базі Spring Boot 3 з використанням трирівневої архітектури (Controller–Service–Repository) , що дозволило чітко відокремити бізнес-правила від шару представлення;

- впроваджено інтелектуальні бізнес-правила та сувору валідацію (Jakarta Bean Validation) на сервісному рівні , які автоматично блокують створення дублікатів (наприклад, реєстрацію з однаковим email або повторний запис клієнта на те саме тренування) та забезпечують безпеку реляційних зв'язків при видаленні об'єктів;

- реалізовано модулі аналітики та моніторингу , що дозволяють групувати заняття за напрямками, виводити рейтинг популярності тренерського складу та відстежувати технічні метрики й стан здоров'я застосунку через Spring Boot Actuator;

- проведено вичерпне функціональне тестування за допомогою інструментів Postman та Swagger UI, у ході якого було верифіковано понад 25 розроблених ендпоінтів та підтверджено коректність роботи централізованого обробника винятків GlobalExceptionHandler.

Під час розробки системи було поглиблено практичні навички застосування ORM-технології Hibernate для налаштування складних зв'язків типу @ManyToOne та @ManyToMany між таблицями бази даних. Набуто досвіду у побудові архітектури REST за допомогою сучасних конструкцій Java Records як незмінних об'єктів перенесення даних (DTO) , реалізації концепції Fail-Fast валідації, а також досвіду розгортання та тестування хмарної інфраструктури додатку з використанням змінних оточення в Postman.

Напрямки подальшого розвитку застосунку:

1. Інтеграція системи автентифікації та авторизації: впровадження Spring Security та протоколу JWT для розмежування прав доступу між ролями клієнта, адміністратора та керівника безпосередньо на рівні API.

2. Розширення фінансового та платіжного функціоналу: додавання сутності «Абонемент» (Subscription) з автоматичним контролем терміну його дії та інтеграція платіжного шлюзу (наприклад, Stripe або LiqPay) для онлайн-оплати занять.

3. Реалізація системи сповіщень: підключення Telegram Bot API або SMS-шлюзів для автоматичного розсилання клієнтам та тренерам нагадувань про майбутні тренування чи зміну розкладу.

4. Масштабування аналітики: розробка розширеного модуля генерації PDF/Excel звітів за обраний період для спрощення бухгалтерського обліку.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. ДСТУ 3008-2015 «Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання».